Développement des composants distribués Web

CHAPITRE 3 :SERVLETCONTEXT, SESSION, EXPRESSION LANGUAGE (EL) ET JAVA SERVER PAGES STANDARD TAG LIBRARY (JSTL)

Paramètres d'initialisation

web.xml

```
<?xml v e r s i on =" 1.0 " encoding ="ISO -8859 -1 " ?>
<web - app >
<servlet >
<servlet - name >ParamInit </ servlet - name >
<servlet - class >ParamInit </ servlet - class >
<init - param >
<param - name >parametre1 </param - name >
<param - value >valeur1 </param - value >
< description >test de parametre </ description >
</init - param >
<init - param >
<param - name >parametre2 </param - name >
<param - value >valeur2 </param - value >
< description >test de parametre </ description >
</init - param >   </ servlet >
<servlet - mapping >
<servlet - name >ParamInit </ servlet - name >
<url - pattern >/ Servlet / ParamInit </url - pattern >
</ servlet - mapping >  </web - app >
```

Paramètres spécifiques à une servlet particulière

Paramètres d'initialisation : récupération grâce à init

ParamInit.java

```
import java .io .*;
import javax . servlet .*;
import javax . servlet . http .*;
p u b l i c c l a s s ParamInit extends HttpServlet {
String param1 ;
String param2 ;
p u b l i c void init () {
t h i s . param1 = getInitParameter (" parametre1 ");
t h i s . param2 = getInitParameter (" parametre2 ");
}
p u b l i c void doGet ( HttpServletRequest requete , HttpServletResponse reponse )
throws
ServletException , IOException {
reponse . setContentType (" text / plain ");
PrintWriter out = reponse . getWriter ();
out . println (" parametre1 = "+ t h i s . param1 );
out . println (" parametre2 = "+ t h i s . param2 );
} }
```

Paramètres d'initialisation : récupération grâce au contexte

web.xml

```
<?xml v e r s i on =" 1.0 " encoding ="ISO -8859 -1 " ?>
<web - app >
<context - param >
<param - name > parametreDeContexte </param - name >
<param - value > valeurContextuelle </param - value >
< description >test de parametre </ description >
</ context - param >
<servlet >
<servlet - name > ParamContext </ servlet - name >
<servlet - class > ParamContext </ servlet - class >
</ servlet >
<servlet - mapping >
<servlet - name > ParamContext </ servlet - name >
<url - pattern >/ Servlet / ParamContext </url - pattern >
</ servlet - mapping >
</web - app >
```

Paramètres communs à toutes les servlets

Paramètres d'initialisation : récupération grâce au contexte

```
ParamContext.java
import java .io .*;
import javax . servlet .*;
import javax . servlet . http .*;
p u b l i c c l a s s ParamContext extends HttpServlet {
p u b l i c void doGet ( HttpServletRequest requete , HttpServletResponse reponse )
throws ServletException , IOException {
ServletContext context = getServletContext ();
String chainecontext = context . getInitParameter (" parametreDeContexte ");
reponse . setContentType (" text / plain ");
PrintWriter out = reponse . getWriter ();
out . println (" Parametre de contexte = "+ chainecontext );
}
}
```

Attributs de contexte

Partager des informations entre plusieurs Servlets :

- Le partage est effectué à travers le contexte d'exécution géré par ServletContext

Obtention du contexte d'une Servlet :

- ServletContext getServletContext(String uripath)

Manipulation d'attributs du contexte par ServletContext :

- void setAttribute(String name, Object o)
- Object getAttribute(String name)
- Enumeration getAttributeNames()
- void removeAttribute(String name)

Suivi de session

Récupération d'une session par HttpServletRequest :

- HttpSession getSession()

Quelques méthodes de HttpSession :

- void setAttribute(String name, Object value)
- void getAttribute(String name)
- void removeAttribute(String name)
- void setMaxInactiveInterval(int secs)
- void invalidate()

Exemple Session

Panier.java

```
public class Panier extends HttpServlet {
        public void doGet (HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException        {
        // Recupere la session
        HttpSession session = request.getSession();
        // Recupere l'age de l'utilisateur
        Age = (int)session.getAttribute("Age");
        if (Age != null) {
            // ... Message d’erreur
                    }
        else {
            // traitement selon l’age
        }   } }
```

Exemple Session

CompteurSession.java

```
public class CompteurSession extends HttpServlet {
private static int compteur = 1;
p u b l i c void doGet ( HttpServletRequest req , HttpServletResponse res)
throws ServletException , IOException {
String message =" Vous êtes éàdj venu ";
HttpSession session = req . getSession ();
if ( session . getAttribute (" DejaVenu ") == null ) {
session . setAttribute (" DejaVenu "," dejavenu ");
message =" Bienvenue ";
compteur ++;}
res . setContentType (" text / html ");
PrintWriter out = res . getWriter ();
out . println ("<head >");
out . println ("<title > Compteur à Session </title >");
out . println (" </head >");
out . println ("<body >");
out . println ("<h1 >"+ compteur +" </h1 >");
out . println ("<h1 >"+ message +" </h1 >");
out . println (" </body >"); }}
```

"Expression language" (EL) en JSP

- Les "expressions languages" (EL) sont des langages de script légers permettant d'évaluer des formules pour manipuler des données et d'accéder aux propriétés d'objets.
  - utilisent des expressions pour modifier des données à la volée (ex: concaténer deux colonnes, convertir des formats de date, appliquer des conditions SI/ALORS)
- Utilisés pour évaluer des expressions retournant des valeurs (chaînes, nombres, booléens).
  - ils permettent de nettoyer les données (suppression de doublons, gestion des valeurs nulles).
- Permettent d'appeler des méthodes, d'accéder aux attributs d'objets (request, session) et d'utiliser des opérateurs.
- Syntaxe
  - Une expression est toujours sous la forme : ${ expression }
  - la valeur retournée par une expression EL positionnée dans un texte ou un contenu statique sera inséré à l'endroit même où est située l'expression

```
<!-- La ligne suivante : -->
<p>12 est inférieur à 8 : ${ 12 lt 8 }.</p>
<!-- Sera rendue ainsi après interprétation de l'expression, 12 n'étant pas inférieur à 8 : -->
<p>12 est inférieur à 8 : false.</p>
```

EL pour la manipulation de données

```
<body>
  <p>
    <!-- Logiques sur des booléens -->
    ${ true && true } <br /> <!-- Affiche true -->
    ${ true && false } <br /> <!-- Affiche false -->
    ${ !true || false } <br /> <!-- Affiche false -->
    <!-- Calculs arithmétiques -->
    ${ 10 / 4 } <br /> <!-- Affiche 2.5 -->
    ${ 10 mod 4 } <br /> <!-- Affiche le reste de la division
entière, soit 2 -->
    ${ 10 % 4 } <br /> <!-- Affiche le reste de la division
entière, soit 2 -->
    ${ 6 * 7 } <br /> <!-- Affiche 42 -->
    ${ 63 - 8 } <br /> <!-- Affiche 55 -->
    ${ 12 / -8 } <br /> <!-- Affiche -1.5 -->
    ${ 7 / 0 } <br /> <!-- Affiche Infinity -->
    <!-- Compare les caractères 'a' et 'b'.-->
    ${ 'a' < 'b' } <br />

    <!-- Compare les chaînes 'hip' et 'hit'. Puisque 'p' < 't',
cette EL affiche false. -->
    ${ 'hip' gt 'hit' } <br />
    <!-- Compare les caractères 'a' et 'b', puis les chaînes
'hip' et 'hit'. Puisque le premier test renvoie true et le
second false, le résultat est false. -->
    ${ 'a' < 'b' && 'hip' gt 'hit' } <br />

    <!-- Compare le résultat d'un calcul à une valeur fixe.
Ici, 6 x 7 vaut 42 et non pas 48, le résultat est false. -->
    ${ 6 * 7 == 48 } <br />
  </p>
  </body>
```

EL pour évaluer des conditions

```
<!-- Vérifications si vide ou null -->
${ empty 'test' } <!-- La chaîne testée n'est pas vide, le résultat est false -->
${ empty '' } <!-- La chaîne testée est vide, le résultat est true -->
${ !empty '' } <!-- La chaîne testée est vide, le résultat est false -->

<!-- Conditions ternaires -->
${ true ? 'vrai' : 'faux' } <!-- Le booléen testé vaut true, vrai est affiché -->
${ 'a' > 'b' ? 'oui'  : 'non' } <!-- Le résultat de la comparaison vaut false, non est affiché -->
${ empty 'test' ? 'vide' : 'non  vide'  } <!-- La chaîne testée n'est pas vide, non vide est affiché -->
```

EL pour manipuler des objets

- il est possible via une expression EL de créer et d'accéder directement à une propriété d'un objet Java.

```
<body>
  <p>
    <!-- Initialisation d'un bean de type Personne avec une action standard, pour l'exemple : -->
    <jsp:useBean id="personne" class="com.exemple.Personne" />
    <!-- Initialisation de sa propriété 'prénom' : -->
    <jsp:setProperty name=" personne " property="prenom" value="MOHAMED"/>
    <!-- Et affichage de sa valeur : -->
    <jsp:getProperty name=" personne " property="prenom" />
  </p>
  </body>
```

Affichage de la chaine MOHAMED

Equivalent à: ${personne.prenom }
Ou ${personne.getPrenom() }

EL pour manipuler des objets

```
<!-- Comparaison d'égalité entre la propriété prenom et la chaîne "Jean-Paul" -->
${ personne.prenom == "Jean-Paul" }

<!-- Vérification si la propriété prenom est vide ou nulle -->
${ empty personne.prenom }

<!-- Condition ternaire qui affiche la propriété prénom si elle n'est ni vide ni nulle, et la chaîne "Veuillez préciser
un prénom" sinon -->
${ !empty personne.prenom ? personne.prenom : "Veuillez préciser un prénom" }
```

EL pour manipuler des objets

- **les expressions EL sont protégées contre un éventuel retour null**

```
<!-- La scriptlet suivante affiche "null" si la propriété "prenom" n'a pas été initialisée,
et provoque une erreur à la compilation si l'objet « personne" n'a pas été initialisé : -->
<%= personne.getPrenom() %>

<!-- L'action suivante affiche "null" si la propriété "prenom" n'a pas été initialisée,
et provoque une erreur à l'exécution si l'objet « personne" n'a pas été initialisé : -->
<jsp:getProperty name=" personne " property="prenom" />

<!-- L'expression EL suivante n'affiche rien si la propriété "prenom" n'a pas été initialisée,
et n'affiche rien si l'objet "personne" n'a pas été initialisé : -->
${personne.prenom }
```

EL pour manipuler des collections

- il est également possible d'accéder aux collections au sens large. Cela inclut donc tous les objets de type java.util.List, java.util.Set, java.util.Map, etc

```
<body>   <p>
    <%
    /* Création d'une liste de légumes et insertion de quatre éléments à l'aide de scriptlet*/
    java.util.List<String> legumes = new java.util.ArrayList<String>();
    legumes.add( "poireau" );
    legumes.add( "haricot" );
    legumes.add( "carotte");
    legumes.add( "pomme de terre" );
    request.setAttribute( "legumes" , legumes );
    %>
    <!-- Les quatre syntaxes suivantes retournent le deuxième élément de la liste de légumes  -->
    ${ legumes.get(1) }<br />
    ${ legumes[1] }<br />
    ${ legumes['1'] }<br />
    ${ legumes["1"] }<br />
  </p>   </body>
```

Désactiver l’interprétation des expressions EL

- à traves une directive
  - **<%@** page isELIgnored ="true" **%>**
- À travers le descripteur de déploiement web.xml
  - **<jsp-config>**
  - **<jsp-property-group>**
  - **<url-pattern>***.jsp**</url-pattern>**
  - **<el-ignored>**true**</el-ignored>**
  - **</jsp-property-group>**
  - **</jsp-config>**

JSTL

- Remplacer les scriptlets JAVA
- La technologie se base sur une bibliothèque de balises (Taglibs) qui Permet d'utiliser des balises JSP comme le JSTL (JavaServer Pages Standard Tag Library)
  - <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
  - **<c:out value="Mon texte" />** : Affiche une valeur (similaire à <%= ... %>).
  - **<c:if test="${condition}"> ... </c:if>** : Exécute le contenu si la condition est vraie.
  - **<c:forEach items="${liste}" var="item"> ... </c:forEach>** : Itère sur une collection.
  - **<c:url value="/chemin/page.jsp"> <c:param name="id" value="123" /> </c:url>** : Crée une URL avec paramètres.
  - **<c:redirect url="/nouvelle/page.jsp" />** : Redirige vers une autre page.

Exemple de code en JSP (taglib)

ProduitServlet.java

```
@WebServlet("/produits")
public class ProduitServlet extends HttpServlet
{
protected void doGet(HttpServletRequest req,
HttpServletResponse resp)
        throws ServletException, IOException {
      req.setAttribute("produits", dao.findAll());

req.getRequestDispatcher("produits.jsp").forwa
rd(req, resp);
 }}
```

produits.jsp

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
 <h2>Liste des produits</h2>
 <table border="1">
   <tr>
      <th>Nom</th>
      <th>Prix</th>
      <th>Action</th>
   </tr>
    <c:forEach items="${produits}" var="p">
      <tr>
         <td>${p.nom}</td>
         <td>${p.prix}</td>
       </tr>
   </c:forEach>
</table>
```

Exercice : annuaire de personnes en JSTL

Développer une application JEE permettant de :

Ajouter une personne dans un annuaire de personnes non persistantes, une personne est identifiée par son non, prénom et son mail.

Lister toutes les personnes de l'annuaire

Rechercher une personne par **nom**

Afficher les résultats avec **JSTL**

Connexion à une base de données

- JDBC : Java Data Base Connectivity: API Java standard permettant aux applications de se connecter, d'interroger et de mettre à jour des bases de données relationnelles via SQL.
- ORM : **Objet-Relationnel Mapping :** permet de lier entre le monde de la programmation **orientée objet** (classes, instances) et celui des **bases de données relationnelles** (tables, lignes, colonnes).
  - Moins de code répétitif à écrire
  - simplifie le changement du SGBD
- Hibernate : framework **ORM** open source pour le langage Java, propose des méthodes assurant l'interaction avec la base de données sans passer par le SQL,
  - crée une "base de données virtuelle" dans votre code. Au lieu d'écrire des requêtes **SQL** manuelles,
- JPA: Java Persistence API : Elle utilise des Entity (classe avec des attributs+accesseurs) annotés pour mapper des objets Java à des tables de base de données.
  - Se base sur Hibernate

Les entités : Entity

- Générées automatiquement à partir du schéma relationnel
- Annoté par @Entity
- Table associée au tuple : @Table(name="FILMS")
  - Pas nécessaire de mettre tous les champs de la table
- Attribut d'une entité = colonne d'une table
  - Changer le nom de la colonne de la BD :
    - @Column(name="nomBD") int nomJava;
- Clé primaire obligatoire : type primitif ou composé
  - @Id
  - @GeneratedValue(strategy=?) : indique comment sont générés les clés :*Auto, Identity, Sequence, Table*

Les entités : Entity

```
@Entity
@Table(name = "FILMS")
public class Film implements java.io.Serializable {
@Id @GeneratedValue(strategy = GenerationType.AUTO)
private int id;
private String name;
public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getName() { return name; }
public void setName(final String name) { this.name = name }
}
```

Gestionnaire de Entity : EntityManager

- Utilisation des EntityBeans : dans des SessionBeans
- L'EntityManager : gestionnaire de persistance ou contexte de persistance.
- Un entity bean est un objet persistant associé à un cycle de vie composé de quatre états:
  - Nouveau (new): un bean qui vient d'être instancié.
  - Managé (Managed): le bean est associé à un contexte de persistance.
    - Suite à une recherche ou une modification
  - Détaché (detached): le bean est détaché du contexte de persistance.
    - Il n'est plus synchronisé à la base
  - Supprimé(removed): le bean est attaché au contexte de persistance mais est prévu pour être supprimé de la base.

```
public class CinemaDAO {
   @PersistenceUnit
    private EntityManagerFactory emf;
 EntityManager em = emf.createEntityManager();
}
```

Gestionnaire de Entity : EntityManager

Principales méthodes d'un EntityManager

Object **find(Class cl, Object key) :**

- Trouve un EntityBean à partir de sa clé primaire

boolean **contains(Object entity) :**

- Vrai si entity est attaché à l'EntityManager

Query **createQuery(String qlString) :**

- Création d'une requête dans le langage « query language »

Query **createNativeQuery(String sqlQuery) :**

- Création d'une requête dans le langage SQL

Gestionnaire de Entity : EntityManager

- void **remove(Object entity) :**
  - Supprime l'entity de la base
- void **refresh(Object entity) :**
  - Recharge le bean à partir de la base

```
public Object findFilm(int id) {
return em.find(Film.class, Integer.valueOf(id));
}
```

```
public void removeFilm(int id) {
em.remove(findFilm(id));
}
```

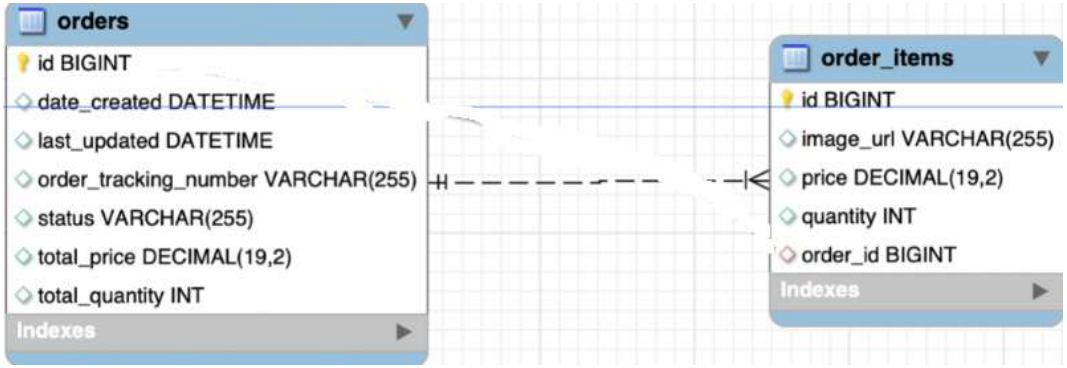
```
public List<Film> findAll() {
    return em.createQuery("SELECT f FROM Film f", Film.class).getResultList();
}
```

Gestionnaire de Entity : EntityManager

- Les associations entre EntityBean
- Cardinalité des relations :
  - 1-1
  - 1-n (ex. : une commande contient n lignes)
  - n-1 (ex. : plusieurs lignes de commandes peuvent concerner le même produit)
  - n-n (ex. : un ou plusieurs cours suivis par plusieurs étudiants)

OneToMany : mapping unidirectionnel

L’association unidirectionnelle OneToMany est plus simple puisque c’est uniquement le côté parent qui définit la relation. Dans le mappage unidirectionnel, nous utilisons uniquement l'annotation @OneToMany.




```
@Entity
@Table(name="order_items")
public class OrderItem {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name="id") private Long id;
//Pas d’attribut de type Order sur OrderItem
```

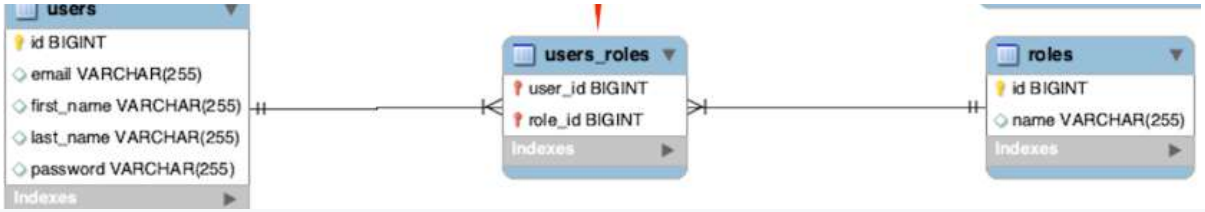
```
@Entity @Table(name="orders")
 public class Order {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name="id")
 private Long id;
// one to many unidirectional mapping // default fetch type for OneToMany: LAZY
@OneToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
@JoinColumn(name = "order_id", referencedColumnName = "id")
 private Set<OrderItem> orderItems = new HashSet<>();
```

OneToMany : mapping bidirectionnel

```
@Entity @Table(name="orders")
public class Order {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
@OneToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER, mappedBy = "order")
 private Set<OrderItem> orderItems = new HashSet<>();
```

```
@Entity
@Table(name="order_items")
public class OrderItem {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
 // default fetch type for ManyToOne: EAGER
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "order_id")
private Order order;
```

ManyToMany




```
@Entity @Table( name = "users" )
 public class User { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private long id;
  @ManyToMany(fetch = FetchType.EAGER, cascade = CascadeType.ALL)
@JoinTable( name = "users_roles", joinColumns = @JoinColumn( name = "user_id", referencedColumnName = "id" ),
inverseJoinColumns = @JoinColumn( name = "role_id", referencedColumnName = "id" ) )
 private Set<Role> roles = new HashSet<>(); }
```

```
@Entity @Table(name = "roles")
public class Role { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private long id;
 private String name;
 @ManyToMany(cascade = { CascadeType.PERSIST, CascadeType.MERGE }, fetch = FetchType.EAGER, mappedBy =
"roles")
 private Set<User> users = new HashSet<>(); }
```

```
void saveRole(){
User user = new User();
 Role roleAdmin = new Role();roleAdmin.getUsers().add(user);
 user.getRoles().add(roleAdmin);  em.persist(roleAdmin); }
```

Exercice : application e-commerce avec panier non persistant

L'application permet de lister et ajouter les catégories de produits proposés par l'application. Les catégories et les produits représentent les entités persistantes de l'application. Ces entités sont projetées respectivement sur deux tables qui portent le même nom d'une base de données relationnelle de type H2. la manipulation de données est effectuée à travers JPA. Une catégorie est caractérisée par un identifiant, un label et une description. Un produit appartient à une seule catégorie. On peut ajouter et lister les produits par catégorie. Un produit est caractérisé par identifiant, une désignation, un prix et une quantité. Le client peut ajouter, si la quantité en stock est suffisante, une quantité d'un produit à son panier non persistant. Si la quantité sélectionnée dépasse celle en stock, l'application lève une exception. Le client peut lister le contenu de son panier.

1. représenter le diagramme de classes de cette application.

2. Créer la base de données relationnelle adéquate.

3. Développer les entités Categorie.java et produit.java représentant la couche persistance. Développer le bean non persistant panier.java.

4. développer les DAO nécessaires

5. développer les servlets qui représentent la couche contrôle.

6. développer les JSP en JSTL représentant la couche présentation.